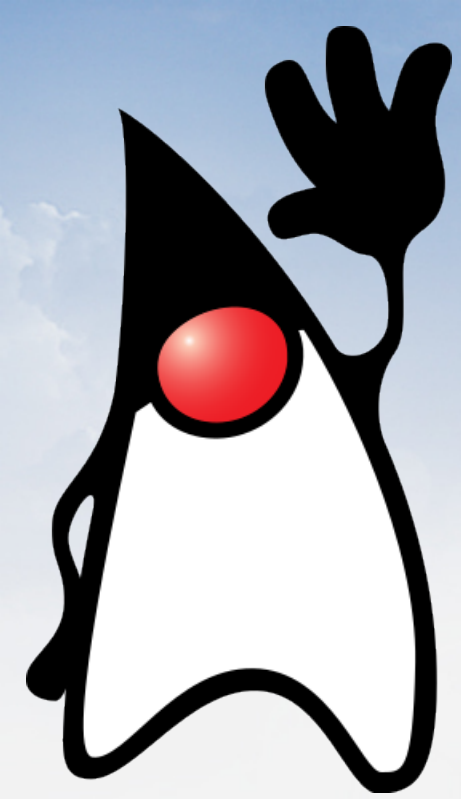




Java Basics1

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Classes in Java

- We will be writing everything inside a Class. You can name the class as you like. Preferably it should start with a capital.
- We will be creating a package before creating a Class inside. The package can have any name also. Preferably it should start with a lowercase letter.
- We will be writing everything under the main method
- If we want to run a file, it should have a main method.
- Only classes which have a main method inside can run.
- If you compile a class/file without main what will happen?

Java

- Java is case sensitive.
- Java compiler ignores white-space characters unless you use them as separators or as components of character constants or string literals.
- Indentation is strongly suggested although it is ignored by compiler.

Simple class with header comment



```
// Author: Safa Bacanli
/* This class salutes class, assignment0
PID: 1231231
NID: xy121212
*/
package assignment0;
public class FirstJavaProgram
{
    public static void main(String[] args)
    {
        System.out.println("Hi Class!");
    }
}
```

comments in
Java are done
with
// for single line
/*comment*/
for multiple lines

For some time,
we will be
writing our code
inside the main
method.

Variable types

- We have primitive and Object data types

- Primitive

- | | | |
|------------------|----------|--|
| • byte | 1 byte | Whole numbers from -128 to 127 |
| • short | 2 bytes | Whole numbers from -32,768 to 32,767 |
| • int | 4 bytes | Whole numbers from -2,147,483,648 to 2,147,483,647 |
| • long | 8 bytes | Whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| • float | 4 bytes | Fractional numbers. Sufficient for storing 6 to 7 decimal digits |
| • double | 8 bytes | Fractional numbers. Sufficient for storing 15 decimal digits |
| • boolean | 1 bit(?) | Stores true or false values |
| • char | 2 bytes | Stores a single character/letter or ASCII values |

- Object type

- **String** ? Stores any kind of numeric and alphanumeric characters



Variable Naming Convention for Java



- The first letters of the variables/methods should be lowercase
- Beginning of each word is capitalized.
- This lowercase/uppercase rule applies for class names but
 - the first letter of class name should be capitalized also.

```
package testPackage;

public class FirstJavaProgram
{
    static int globalVariable;
    public static void main(String[] args){
        int numberOfStudentsInClassroom = 45;
        System.out.println("Hi Class!");
    }

    public static int someFunction(){        return 1;    }
}
```

This naming convention is called camelCase.


```
package test1;
public class test{

    public static void main(String[] args){
        int age = 45;
        String name = "John Doe";
        double price = 34.123873263;
        long longNumber = 12341234123l;
        float val = 3.4f;
        System.out.println( name );

        age = age + 1; // age++; or ++age;
    } //end of main method
}
```

long data type

If you don't specify the letter “l” after the number, java will think it is integer

You get

error: integer number too large

float data type

If you don't specify the letter “f” after the number, java will think it is double.

Therefore you are trying to put double inside float.

You get

error : incompatible types: possible lossy conversion from double to float

```
// This class shows the integer overflow example
```

```
package test1;
```

```
public class test{
```

```
    public static void main(String[] args){
```

```
        int maximum = Integer.MAX_VALUE;
```

```
        System.out.println("Maximum value that can be stored in integer data type is");
```

```
        System.out.println( maximum ); // we can write the variable name inside println
```

```
        maximum++; // maximum = maximum + 1
```

```
        System.out.println("after maximum is increased by 1, maximum is");
```

```
        System.out.println( maximum);
```

```
    } //end of main method
```

```
}
```


++/--

- age++ and ++age is same if they are used by themselves.
- If they are made to set a variable, they will behave differently.

```
int age = 9;  
age++; // age = age + 1; => age = 10  
int updated = ++age;  
//Code below are same with the above  
// age = age + 1;  
// int updated = age;  
System.out.println(age+"\t"+updated);
```

11 11

```
int age = 9;  
++age; // age = age + 1; => age = 10  
int updated = age++;  
//Code below are same with the above  
//int updated = age;  
//age = age + 1;  
System.out.println(age+"\t"+updated);
```

11 10

The same mentality exists for -- also (number--; => number = number - 1;)

Output Statements

- `System.out.print()` does not pass to the next line
- `System.out.println()` passes to next line

```
System.out.print("One line");  
System.out.println("Second line");
```

One lineSecond line

```
System.out.println("One line");  
System.out.println("Second line");
```

One line
Second line

```
System.out.println(someStr);      =>    System.out.print(someStr+"\n");
```

Printing Strings

- Escape sequences exist in Java also.
- `\\` => `\`, `\\\\` => `\\`
- `\n` => newline
- `\t` => tab

```
System.out.print("test\ttab after\n");  
System.out.println("either yes\\no");
```

```
test    tab after  
either yes\nno
```

```
System.out.println(1+6);  
System.out.println(1 +""+6);  
System.out.println(1+6+"");  
System.out.println("" +1+6);
```

```
7  
16  
7  
16
```

System.out.printf()

- For printing and string formatting
 - Probably generated for C programmers
 - Padding and string alignment can be done with printf
 - DO NOT USE THIS UNLESS YOU NEED A PADDING.

```
package test1;
public class test{
    public static void main(String[] args){
        String name = "john";
        int age = 96;
        System.out.printf("%f%n", 5.1473); // print as float
        System.out.printf("'%s' is '%d' years old. %n", name, age);
        System.out.printf("%2.2s", "Hi there!"); //gives padding
    } //end of main method
}
```

```
5.147300
'john' is '96' years old.
  Hi there!
```

%d	Signed decimal integer
%f	Floating point
%o	Octal number
%c	Character
%s	String
%i	Integer base 10
%u	Unsigned decimal number
%x	Hexadecimal Number
%%	Percentage
%n	Newline (sounds like \n)

-	Justify left
+	Need to show + or minus in the formatted string
^	Uppercase
0	Zero padded numeric values

The Scanner Class

- Scanner class in Java allows us to read input from the keyboard
- Part of the util package (contains the Scanner Class)
 - This means you will need to import it in your program!
 - `import java.util.Scanner;`

Method	What it does?
<code>nextBoolean()</code>	Reads a Boolean
<code>nextDouble()</code>	Reads a double
<code>nextFloat()</code>	Reads a float
<code>nextInt()</code>	Reads an int
<code>nextLine()</code>	Reads a String

Scanner

```
package scannerTest;
import java.util.Scanner;
public class HelloWorld {
    public static void main(String[] args) {
        Scanner scan=new Scanner(System.in);
        String word=scan.next(); //gets everything until user presses Space
        System.out.println("word is "+word);
        scan.nextLine(); // consume the new line (Remove this line and test)

        String givenLine = scan.nextLine();
        //gets everything until user presses Enter
        System.out.println("Line is " + givenLine);

        int givenInt = scan.nextInt(); // reads input as int else gives error
        System.out.println("Int is " + givenInt);

        double givenDouble = scan.nextDouble(); // reads inside double
        System.out.println("double is "+givenDouble);
        scan.close();// close the input Stream
    } //end of main method
}
```

after using next() if you use nextLine() you will have an input problem.

Either always use *nextLine()* then parse the String with parseInt/parseDouble

or

consume the new line character by calling

scan.nextLine();

after each

scan.nextInt(),

scan.next() or

*scan.next**

<https://stackoverflow.com/questions/13102045/scanner-is-skipping-nextline-after-using-next-or-nextfoo>

Arithmetic in Java

- Like many programming languages, Java has arithmetic operators which can be used to write expressions.
- Arithmetic Operators in Java
 - +
 - -
 - *
 - /
 - %
- Important: The modulus operator works for both integers and real numbers.

Modulus and arithmetic code examples

```
package negativeMod;

public class NegativeModulus {
    public static void main(String[] args) {
        int a = -10;
        int b = 3;
        System.out.println(a/b); // int/int result is int
        int dividedNum = a / b;
        float addedNum = a / b +1; // => (a/b) + 1 is converted to float

        //Printing two numbers with a space between each other
        System.out.println(dividedNum+" "+addedNum);
        System.out.println("-10 % 3 = " + (a % b)); // Output: -10 % 3 = -1
        System.out.println("10 % -3 = " + (10 % -3)); // Output: 10 % -3 = 1

    } // end of main method
}
```

-3
-3 -2.0
-10 % 3 = -1
10 % -3 = 1

The sign of the result follows the dividend (a).

Exercises

1) Get 2 numbers from the User (let us say x and y) and print

Addition

Subtraction

Division

Multiplication and

Mod of those numbers.

2) Try different combinations with different types of variables to add/subtract/divide/multiply them and see the result.

3) Divide some number by zero. Divide zero by zero. Print the results.

4) As an extension also print $\log_x y$ and $\log_y x$ and $\text{power}(x^y)$ of those numbers.

Math Class

- The Math class contains mathematical methods that you can utilize.
- The Math class is stored in the language package. **What does that mean?**

Method	What it does?
<code>pow(x,y)</code>	x^y
<code>sin(x)</code>	$\sin(x)$, x in radians
<code>sqrt(x)</code>	returns the square root of a number
<code>round(x)</code>	Rounds to nearest integer
<code>exp(x)</code>	e^x
<code>log(x)</code>	$\ln(x)$
<code>log10(x)</code>	$\log_{10}(x)$

Constants	What it represents?
PI	Math.PI
E	Math.E

- NaN – Not a Number
- <https://docs.oracle.com/javase/8/docs/api/java/lang/Math.html>

```
package mathTest;
```

```
public class mathTest{
```

```
    public static void main(String[] args){  
        double pi = Math.PI;  
        double e = Math.E;
```

```
        double result=Math.sin(90);  
        System.out.println(result);
```

```
        double result = Math.sin(pi/2);  
        System.out.println(result);
```

```
        result = Math.pow(4,2);  
        System.out.println(result);  
        result=Math.pow(4,-1);  
        System.out.println(result); //0.25  
        System.out.println(Math.sqrt(4)); // 2.0
```

```
    }
```

```
}
```

We do not import
any library to use
Math functions

This is in
radians. not
degrees!

This will give
error. *result* is
defined before!



Random Number Generation

```
package randomTester;
import java.util.Random; // We need to write this at the top before class
public class RandomTest{

    public static void main(String[] args){
        Random random = new Random(); // We need to initialize random
        int upperbound = 6;
        int intRandom = random.nextInt(upperbound); // [0, upperbound)
        double doubleRandom = random.nextDouble(); //[0,1)
        float floatRandom = random.nextFloat();// [0,1)

        // Printing the generated random numbers
        System.out.println("Random die value " + (intRandom + 1)); // [1,7)
        System.out.println("Random float value between 0.0 and 1.0 : " + floatRandom);
        System.out.println("Random double value between 0.0 and 1.0 : " + doubleRandom);
    } //end of main method
} //end of class
```

If you give a long number to Random as a parameter, you can have seeded random. Random(4) is possible.

Why did we add 1?

Error Types

- Compile time
 - Syntactically incorrect code
 - `float f=12.4`
- Runtime
 - if file not found
 - `numberFromUser/0`
 - `float f = Float.parseFloat("test");`
- Logical error
 - Everything works but not correctly

package,import,class

- The first statement of a Java class should be package
 - In this course do NOT use default package (meaning no package statement).
- Then imports should be written
 - For Standard Java libraries we use import java.sth..
- Then the class name should be written
 - public class HelloWorld
 - The filename should EXACTLY match with class name
 - HelloWorld.java file should have public class HelloWorld
 - Java is case sensitive!